PreCompile: Reconnaissance & Setup Report

SPEC: IDF-B_PreCompile_Final.pdf | STATUS: Recon & Skeleton Complete | DATE: October 17, 2026

This document consolidates the complete output of the reconnaissance, environment probing, specification breakdown, repository skeleton setup, and 11-step implementation roadmap for **PreCompile** (an anticipatory, confidence-gated scheduling priority reservation system using learned event-to-process latency for Linux).

1. Environment Detection & Subsystem Analysis

Probing commands were executed directly inside the active Linux subsystem (**Ubuntu on WSL2**):

Raw Subsystem Probe Output

```
$ uname -r
6.18.33.2-microsoft-standard-WSL2

$ cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04 (Resolute Raccoon)"
ID=ubuntu

$ systemd-detect-virt
wsl

$ stat -fc %T /sys/fs/cgroup
cgroup2fs

$ cat /sys/fs/cgroup/cgroup.controllers
cpuset cpu io memory hugetlb pids rdma

$ cat /proc/pressure/cpu
some avg10=3.27 avg60=0.72 avg300=0.15 total=522104
full avg10=0.00 avg60=0.00 avg300=0.00 total=0

$ cat /proc/sys/fs/inotify/max_user_watches
524288

$ python3 --version; gcc --version | head -1; make --version | head -1
Python 3.14.4
sh: 1: gcc: not found
sh: 1: make: not found
```

Subsystem Assessment & Required Fallbacks

Subsystem / Property	Observed Value	Design Impact / Limitation	Required Fallback Strategy
Virtualization Platform	`wsl` (WSL2)	Cross-OS mounts (`/mnt/c/...`) use 9P/drvfs, introducing filesystem latency and potential inotify event lag for Windows-hosted files. `systemd` init may also be disabled.	**Fallback:** Benchmarking and project registration should reside on native ext4 mounts (e.g. `~/projects/...`). Support direct background daemon execution (PID-based) in addition to systemd unit files.
Cgroup Hierarchy	`cgroup2fs`	**Fully supported:** Unified cgroup v2 hierarchy is enabled and mounted.	None needed.
CPU Controller	Present (`cpu`)	`cpu.weight` manipulation is supported by the kernel, but requires `root` or delegated cgroup permission.	**Fallback:** Daemon runs as `root` (or with user delegation); without privilege, degrades gracefully to observation-only mode per Stage 10.
CPU Pressure (PSI)	Supported (`some` avg10/60/300)	`/proc/pressure/cpu` is active and exporting valid contention metrics.	None needed.
Inotify Watch Limit	`524,288` watches	Very generous limit; easily monitors extensive source trees.	None needed.
Toolchain	`gcc` & `make` missing	Target C/C++ projects cannot be compiled without installing build tools.	**Fallback:** Install `build-essential` and `stress-ng` prior to running Stage 11 benchmarks.
Python Environment	Python 3.14.4	Excellent choice: native access to `ctypes` , `os` , `sqlite3` , `asyncio` , and file-based cgroup manipulation without native build overhead.	None needed.

2. System Specification Summary (SPEC_SUMMARY.md)

2.1 System Architecture & Stage Mapping

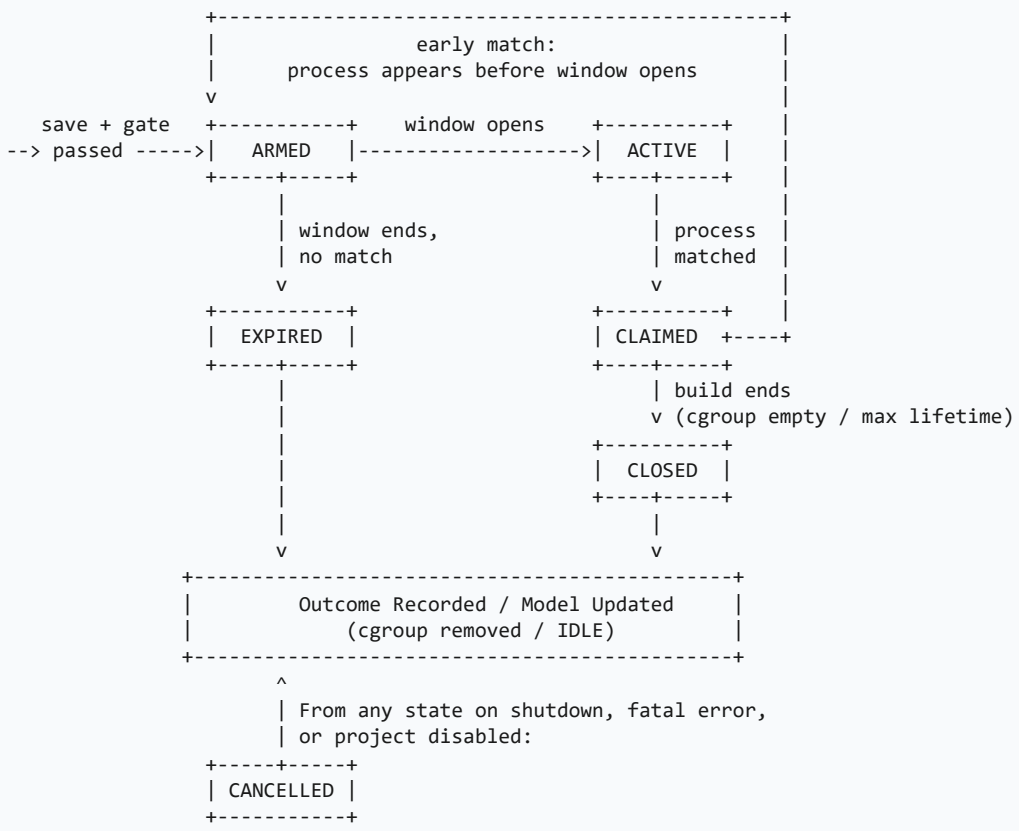
Stage	Name	Key Responsibilities	Module
Stage 1	Event Detection	Watches project roots using inotify (IN_CLOSE_WRITE, IN_MOVED_TO); debounces rapid save bursts (0.15s window); filters VCS/temp files; suppresses saves while claimed build is running.	`precompile/monitor.py`
Stage 2	Process-Creation Observation	Detects process spawns via /proc polling (first pass) or netlink proc connector (cn_proc); matches executables against tool profiles; validates resolved working directories; tracks process hierarchy.	`precompile/watcher.py` `precompile/matcher.py`
Stage 3	Event-Conditioned Latency Learning	Records delay *x* between save and matching build; discards delays >30s; tracks running EWMA mean and variance per (project, file_class, tool_profile); applies 3-sigma outlier clipping.	`precompile/predictor.py` `precompile/store.py`
Stage 4	Confidence Gate & Prediction Window	Calculates prediction window [mean - 2σ - 0.3s, mean + 2σ + 0.3s]; enforces confidence gate (≥5 samples, CV ≤ 0.5 or σ ≤ 0.25s, rolling hit-rate ≥ 0.6 over last 10 once ≥5 exist).	`precompile/predictor.py` `precompile/gates.py`
Stage 5	Contention-Gated, Cost-Aware Arming	Reads /proc/pressure/cpu (PSI); evaluates expected benefit against expected cost of false reservation; arms reservation only under contention.	`precompile/gates.py`
Stage 6	Anticipatory Reservation	Allocates empty cgroup v2 with elevated cpu.weight before build process exists; sets state to ARMED; resets timer on newer saves; limits concurrency (max 1 per project).	`precompile/reservation.py` `precompile/cgroup.py`
Stage 7	Identity-Bound Claiming	Intensifies scan during ACTIVE window; atomically claims matching process via test-and-set flag (supports early ARMED-to-CLAIMED); moves PID and existing children into cgroup.	`precompile/reservation.py` `precompile/matcher.py` `precompile/cgroup.py`
Stage 8	Expiry, Release & Feedback	Governs lifecycle (IDLE, ARMED, ACTIVE, CLAIMED, EXPIRED, CLOSED, CANCELLED); releases cgroup when empty or on window expiry; feeds delays, hits, and misses to learning engine.	`precompile/reservation.py` `precompile/store.py`
Stage 9	Self-Measured Benefit	Withholds random share of reservations (holdouts); compares build durations at matched contention levels; scales weight down or stops arming if no speedup is realized.	`precompile/benefit.py`

Stage	Name	Key Responsibilities	Module
Stage 10	Safety, Bounded Harm & Fallback	Prohibits real-time scheduling classes (CFS only); enforces weight caps (≤500), max boost duration, hourly quotas; cleans stale cgroups; degrades to observation-only without privilege.	`precompile/safety.py` `precompile/config.py`

2.2 Exact Predictor Equations & Thresholds

> **1. Correlation Window & Outlier Clipping:**
> Accept delay *x* only if: $x \leq 30.0\ s$.
> Clip outlier before EWMA update: $x_{clipped} = \max(\mu - 3\sigma, \min(x, \mu + 3\sigma))$ (once ≥ 5 samples exist).
>
> **2. EWMA Mean & Variance Formulation (α = 0.2):**
> $\mu_n = (1 - \alpha) \cdot \mu_{n-1} + \alpha \cdot x_{clipped}$
> $var_n = (1 - \alpha) \cdot (var_{n-1} + \alpha \cdot (x_{clipped} - \mu_{n-1})^2)$
> $\sigma_n = \sqrt{var_n}$
>
> **3. Reservation Prediction Window (k = 2.0, m = 0.3s):**
> $Window = [\ \max(0, \mu_n - 2\sigma_n - 0.3s),\ \mu_n + 2\sigma_n + 0.3s\ ]$
>
> **4. Confidence Gate Thresholds:**
> • Sample Count: $N \geq 5$
> • Variability: $CV = \sigma / \mu \leq 0.5$ OR $\sigma \leq 0.25\ s$ (for very small means)
> • Recent Hit-Rate: $Hit\text{-}Rate \geq 0.6$ over last 10 predictions (once ≥ 5 predictions logged)

2.3 Reservation State Machine



2.4 Process Claiming & Safety Rules

- **Identity Criteria:** Executable in project tool profile (`gcc` , `clang` , `make` , etc.); resolved working directory (via `realpath`) within project root; not already in a reserved cgroup.
- **Atomic Claim:** Atomic test-and-set claim flag ensures exactly one process claims the reservation.
- **Ancestor Hierarchy:** If multiple candidates appear concurrently, the topmost ancestor within the project is claimed; existing children are migrated into the cgroup; future children inherit automatically.
- **Safety Invariants:** Real-time scheduling (`SCHED_FIFO` , `SCHED_RR`) is strictly forbidden; CPU weight is capped (`cpu.weight ≤ 500`); boosts have bounded maximum lifetime; stale cgroups are purged at startup.

2.5 Validation Protocol (Section 8)

- **Compared Conditions:** `C0` (no system/baseline), `C1` (reactive boost after spawn), `C2` (full PreCompile), `C3` (static always-on boost).
- **5 Ablation Studies:** (A1) Reactive boost; (A2) Without confidence gate; (A3) Without contention gate; (A4) Fixed static window; (A5) Without randomized holdouts.
- **Trigger Scenarios:** (A) Automatic build after save; (B) Irregular manual build (gate abstains); (C) Save without build (clean expiry).
- **Key Metrics:** Build completion time (median and 95th percentile), prediction hit-rate, false-reservation rate, claim latency, CPU overhead. Evaluated over ≥50 repeated interleaved runs per condition with Mann-Whitney U significance testing.

3. Repository Skeleton & Design Decisions

3.1 Repository Structure

```
precompile/
├── cli.py          # Entry point (daemon, status, projects, predict) - Stages 1-10
├── config.py       # AppConfig, thresholds, safety limits - Stage 10
├── monitor.py      # inotify save event watcher & debouncing - Stage 1
├── watcher.py      # Process creation detection (/proc, cn_proc) - Stage 2
├── matcher.py      # Executable tool & project root matcher - Stages 2 & 7
├── store.py        # SQLite database layer - Stages 3, 4, 8, 9
├── predictor.py    # EWMA latency learning & window calculation - Stages 3 & 4
├── gates.py        # Confidence gate & PSI CPU contention gate - Stages 4 & 5
├── reservation.py # Reservation state machine & atomic claim - Stages 6, 7, 8
├── cgroup.py       # Linux cgroup v2 controller - Stages 6 & 7
├── benefit.py      # Randomized holdouts & self-measured benefit - Stage 9
└── safety.py       # Harm bounding, budget caps & privilege checks - Stage 10
tests/
├── __init__.py
└── test_skeleton.py # Structural verification suite
pyproject.toml, README.md, PLAN.md, .gitignore
```

3.2 Recorded Design Decisions

- **Single CLI entry point:** Unified `precompile` CLI provides subcommands: `daemon`, `status`, `projects`, and `predict`.
- **/proc polling in first pass:** Process detection uses `/proc` table polling initially, leaving netlink process connector (`cn_proc`) as a zero-latency later upgrade.
- **SQLite for dynamic data, YAML for settings:** SQLite (`precompile.db`) stores projects, latency samples, prediction records, and holdout history. `config.yaml` is reserved exclusively for static settings.
- **Root execution with graceful degradation:** Daemon runs as `root` for cgroup v2 operations. When run unprivileged, it degrades to observation-only mode without crashing (Stage 10).
- **Observation-first implementation:** Steps 1 to 6 focus strictly on observation, learning, and gating without applying kernel scheduling modifications.

3.3 Skeleton Verification Output

```
$ python3 -m unittest discover -s tests
..
----------------------------------------------------------------------
Ran 2 tests in 0.001s

OK
```

4. Implementation Roadmap (PLAN.md)

Step	Deliverable	Verification Protocol
1. Recon + Skeleton	Environment probe, `SPEC_SUMMARY.md`, repository skeleton with stage-mapped Python modules, type signatures, and tests.	`python3 -m unittest discover -s tests` confirms clean imports and `NotImplementedError` stubs.
2. File Monitor	`precompile/monitor.py`: inotify watches, 0.15s debouncing to last timestamp, ignore filters (.git, build dirs), save suppression during builds.	`tests/test_monitor.py`: simulates save bursts, asserts 1 debounced event with final timestamp, verifies ignore patterns and suppression.
3. Process Watcher + Matcher	`precompile/watcher.py` (/proc polling at 10ms) & `precompile/matcher.py` (tool binary match, realpath cwd inside project, topmost ancestor resolution).	`tests/test_watcher.py` & `test_matcher.py`: spawn subshell commands, assert PID/cwd extraction, out-of-tree rejection, ancestor tree resolution.
4. SQLite Store	`precompile/store.py`: schema for projects, delay samples, prediction outcomes, holdouts; WAL mode; rolling 10-sample hit-rate queries.	`tests/test_store.py`: project CRUD, sample logging, rolling hit-rate calculation under temporary SQLite database.
5. Predictor + Confidence Gate + Window	`precompile/predictor.py` (EWMA α=0.2, 3σ clip, 30s window) & `precompile/gates.py` (≥5 samples, CV≤0.5 or σ≤0.25s, hit-rate≥0.6).	`tests/test_predictor.py` & `test_gates.py`: verify EWMA against known vector, 3σ clipping, correlation discard, gate abstention on high variance.
6. Daemon + CLI Observation Mode	Observation-only daemon in `precompile/cli.py` integrating monitor, watcher, matcher, store, predictor; CLI subcommands (daemon, status, projects, predict).	Register mock project, trigger simulated saves and builds with fixed delays; verify via `precompile status` that EWMA converges without root.
7. Arming Gate (PSI, Cost/Benefit, Holdout)	PSI reader parsing `/proc/pressure/cpu`, contention threshold, expected benefit vs. cost model, randomized 10% holdout selector.	`tests/test_gates.py`: mock idle and high CPU pressure inputs; verify uncontented systems reject arming; verify 10% holdout distribution across 1000 trials.
8. Reservation Manager + Cgroup Controller	`precompile/cgroup.py` (cgroup2fs, elevated cpu.weight=500, cgroup.procs migration, emptiness poll) & `precompile/reservation.py` (state machine, early ARMED-to-CLAIMED, atomic claim).	`tests/test_reservation.py` & `test_cgroup.py`: state machine transitions, early claim in ARMED, atomic claim race tests, cgroup creation/removal.
9. Benefit Estimator	`precompile/benefit.py`: matched-load comparison of reserved vs. holdout build times, Mann-Whitney U test, dynamic weight downward scaling.	`tests/test_benefit.py`: synthetic datasets with speedup (weight maintained) vs. zero speedup (weight decreased and arming disabled).
10. Safety + Systemd + Ablation Flags	`precompile/safety.py`: no-realtime invariant, weight cap ≤500, max duration, hourly quota, stale cgroup cleaning, non-root	`tests/test_safety.py`: hourly budget exhaustion test, non-root fallback warning

Step	Deliverable	Verification Protocol
	fallback; systemd service; CLI ablation flags (A1-A5).	test, stale cgroup scrubber test, ablation flag toggling.
11. Evaluation Harness (C0-C3, Ablations, Concurrency)	Benchmark test harness in `benchmarks/` running ≥50 interleaved runs across C0-C3, 5 ablations, under stress-ng load; concurrency & holdout validation tests.	Run automated test matrix; assert C2 median build time < C0/C1 under contention, zero multiple-claim races in concurrency test, report p-values.